FACULTY ASSIGNED TASK 57 · SECTION H

LAB 7 / FUZZ TESTING

Property-based testing and input validation hardening

ARJUN SRIKANTH

PES1UG24AM920 • 12 November 2026

3	19	29
parsers tested	original failures	final passing tests

Assigned repository

SETAPESU26 / L6_L7_57_income_tax

Bugs found and fixes applied

clean_pan	Failure: None / non-string values crashed; whitespace was not fully normalized. Fix: Type-check, return empty string when invalid, normalize all whitespace.
parse_amount	Failure: Malformed rupees, currency prefixes, negative values, empty input crashed. Fix: Strict rupee pattern, number bounds, and None for invalid amounts.
parse_tax_slab	Failure: Missing separators, reversed bounds and invalid tax rate crashed or passed unchecked. Fix: Validate LOW-HIGH:RATE structure and limits, otherwise return None.

Execution and reproducibility

The original faculty module was run with malformed input, then Hypothesis and example-based tests were executed against the unchanged starter. After making the defensive fixes, the same test file was executed again, with all 29 tests passing.

Commands: python income_tax_parser.py | python -m pytest -q test_income_tax_parser.py

Evidence integrity: the screenshots in this PDF are legible renders of the actual captured terminal stdout/stderr, not photographs of a terminal window. Full raw logs are committed in the Lab-7/evidence folder; the failing-test image shows a cropped portion of that log.

EVIDENCE 01 AND 03

APPLICATION INPUT HANDLING

The same malformed-input scenario before and after the fix

SS1 • Original script crashes on malformed rupee input

Unchanged faculty code. A valid PAN is entered, followed by invalid amount text. The script terminates with ValueError before asking for a tax slab.

Before fix – buggy application

```
$ python income_tax_parser.py (PAN valid; amount=not money)
==== Running clean_pan ====
Enter a PAN (e.g. abcde 1234f): Cleaned PAN: ABCDE1234F

==== Running parse_amount ====
Enter an amount (e.g. Rs 12,50,000): Traceback (most recent call last):
  File "/private/tmp/se-labs-920/submission/Lab-7/income_tax_parser.py", line 39, in <module>
    main()
  File "/private/tmp/se-labs-920/submission/Lab-7/income_tax_parser.py", line 32, in main
    print("Amount in rupees:", parse_amount(value))
                               ^^^^^^^^^^^^^^^^^^^
  File "/private/tmp/se-labs-920/submission/Lab-7/income_tax_parser.py", line 14, in parse_amount
    return int(text)
           ^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'not money'
```

Captured command output · rendered without editing test results

SS3 • Fixed script handles malformed input without crashing

The same PAN and amount are entered; invalid amount and tax slab are handled gracefully and displayed as clear validation results.

After fix – invalid inputs handled

```
$ python income_tax_parser.py (PAN valid; amount=not money; slab=invalid slab)
==== Running clean_pan ====
Enter a PAN (e.g. abcde 1234f): Cleaned PAN: ABCDE1234F

==== Running parse_amount ====
Enter an amount (e.g. Rs 12,50,000): Amount in rupees: Invalid amount

==== Running parse_tax_slab ====
Enter a tax slab as LOW-HIGH:RATE (e.g. 250000-500000:5): Tax slab (low, high, rate): Invalid tax slab
```

Captured command output · rendered without editing test results

EVIDENCE 02

FUZZING FINDS REAL BUGS

Hypothesis and malformed input examples against unchanged starter

SS2 • Failing pytest run on the original implementation

The screenshot displays the start and failure summary of the original run. Full captured output is stored in evidence/SS2_failing_fuzz_tests_terminal.txt.

Before fix – failing Hypothesis tests

```
$ python -m pytest -q --tb=line test_income_tax_parser.py
=== Original faculty program: expected RED test run ===
FFFF.F.F.FF..FFFFF....FFFFFF.                                            [100%]
================================== FAILURES ===================================
E   AttributeError: 'NoneType' object has no attribute 'strip'

During handling of the above exception, another exception occurred:
E   Failed: clean_pan crashed on None: AttributeError("'NoneType' object has no attribute 'strip'")
    Failing explicit example: test_clean_pan_no_crash(
        s=None,
[... middle tracebacks cropped for legibility; full raw log saved ...]
/private/tmp/se-labs-920/baseline_lab7/income_tax_parser.py:21: ValueError: not enough values to unpack (expected 2, got 1)
E   ValueError: not enough values to unpack (expected 2, got 1)
/private/tmp/se-labs-920/baseline_lab7/income_tax_parser.py:21: ValueError: not enough values to unpack (expected 2, got 1)
E   AttributeError: 'NoneType' object has no attribute 'split'
/private/tmp/se-labs-920/baseline_lab7/income_tax_parser.py:21: AttributeError: 'NoneType' object has no attribute 'split'
=========================== short test summary info ===========================
FAILED test_income_tax_parser.py::test_clean_pan_no_crash - Failed: clean_pan...
FAILED test_income_tax_parser.py::test_parse_amount_no_crash - Failed: parse_...
FAILED test_income_tax_parser.py::test_parse_tax_slab_no_crash - Failed: pars...
FAILED test_income_tax_parser.py::test_non_string_types_never_crash - Excepti...
FAILED test_income_tax_parser.py::test_clean_pan_examples[  abcde\t1234f \n-ABCDE1234F]
FAILED test_income_tax_parser.py::test_clean_pan_examples[None-] - AttributeE...
FAILED test_income_tax_parser.py::test_parse_amount_examples[rs. 1,234-1234]
FAILED test_income_tax_parser.py::test_parse_amount_examples[\u20b9 2,50,000-250000]
FAILED test_income_tax_parser.py::test_parse_amount_examples[Rs -50-None] - A...
FAILED test_income_tax_parser.py::test_parse_amount_examples[1,2,3-None] - As...
FAILED test_income_tax_parser.py::test_parse_amount_examples[-None] - ValueEr...
FAILED test_income_tax_parser.py::test_parse_amount_examples[not money-None]
FAILED test_income_tax_parser.py::test_parse_amount_examples[None-None] - Att...
FAILED test_income_tax_parser.py::test_parse_tax_slab_examples[500000-250000:5-None]
FAILED test_income_tax_parser.py::test_parse_tax_slab_examples[100-200:101-None]
FAILED test_income_tax_parser.py::test_parse_tax_slab_examples[10-20:three-None]
FAILED test_income_tax_parser.py::test_parse_tax_slab_examples[100 - 200-None]
FAILED test_income_tax_parser.py::test_parse_tax_slab_examples[-None] - Value...
FAILED test_income_tax_parser.py::test_parse_tax_slab_examples[None-None] - A...
19 failed, 10 passed in 0.65s
```

Captured command output · rendered without editing test results

RESULT

19 failed, 10 passed. All three parser functions had reproducible failures.

EVIDENCE 04

FUZZING REGRESSION TESTS

Final test suite against corrected input parsers

SS4 • Complete test suite passing after fixes

The same test file covers hypothesis-generated malformed inputs, explicit edge cases, and round-trip properties for valid amounts and slabs.

After fix – all 29 tests pass

```
============================= test session starts ==============================
platform darwin -- Python 3.12.15, pytest-9.2.0, pluggy-1.7.0 -- /private/tmp/se-labs-920/.venv/bin/python
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: /private/tmp/se-labs-920/submission/Lab-7
plugins: hypothesis-6.170.15
collecting ... collected 29 items

test_income_tax_parser.py::test_clean_pan_no_crash PASSED                [  3%]
test_income_tax_parser.py::test_parse_amount_no_crash PASSED             [  6%]
test_income_tax_parser.py::test_parse_tax_slab_no_crash PASSED           [ 10%]
test_income_tax_parser.py::test_non_string_types_never_crash PASSED      [ 13%]
test_income_tax_parser.py::test_clean_pan_examples[abcde 1234f-ABCDE1234F] PASSED [ 17%]
test_income_tax_parser.py::test_clean_pan_examples[  abcde\t1234f \n-ABCDE1234F] PASSED [ 20%]
test_income_tax_parser.py::test_clean_pan_examples[-] PASSED             [ 24%]
test_income_tax_parser.py::test_clean_pan_examples[None-] PASSED         [ 27%]
test_income_tax_parser.py::test_parse_amount_examples[Rs 12,50,000-1250000] PASSED [ 31%]
test_income_tax_parser.py::test_parse_amount_examples[rs. 1,234-1234] PASSED [ 34%]
test_income_tax_parser.py::test_parse_amount_examples[\u20b9 2,50,000-250000] PASSED [ 37%]
test_income_tax_parser.py::test_parse_amount_examples[0-0] PASSED        [ 41%]
test_income_tax_parser.py::test_parse_amount_examples[  25  -25] PASSED  [ 44%]
test_income_tax_parser.py::test_parse_amount_examples[Rs -50-None] PASSED [ 48%]
test_income_tax_parser.py::test_parse_amount_examples[1,2,3-None] PASSED [ 51%]
test_income_tax_parser.py::test_parse_amount_examples[-None] PASSED      [ 55%]
test_income_tax_parser.py::test_parse_amount_examples[not money-None] PASSED [ 58%]
test_income_tax_parser.py::test_parse_amount_examples[None-None] PASSED  [ 62%]
test_income_tax_parser.py::test_parse_amount_round_trip PASSED           [ 65%]
test_income_tax_parser.py::test_parse_tax_slab_examples[250000-500000:5-expected0] PASSED [ 68%]
test_income_tax_parser.py::test_parse_tax_slab_examples[ 0 - 250000 : 0 -expected1] PASSED [ 72%]
test_income_tax_parser.py::test_parse_tax_slab_examples[250000-500000:100-expected2] PASSED [ 75%]
test_income_tax_parser.py::test_parse_tax_slab_examples[500000-250000:5-None] PASSED [ 79%]
test_income_tax_parser.py::test_parse_tax_slab_examples[100-200:101-None] PASSED [ 82%]
test_income_tax_parser.py::test_parse_tax_slab_examples[10-20:three-None] PASSED [ 86%]
test_income_tax_parser.py::test_parse_tax_slab_examples[100 - 200-None] PASSED [ 89%]
test_income_tax_parser.py::test_parse_tax_slab_examples[-None] PASSED    [ 93%]
test_income_tax_parser.py::test_parse_tax_slab_examples[None-None] PASSED [ 96%]
test_income_tax_parser.py::test_tax_slab_round_trip PASSED               [100%]

============================== 29 passed in 0.18s ==============================
```

Captured command output · rendered without editing test results

RESULT

29 passed. No failing tests or unhandled exceptions.

REFLECTION / THREE QUESTIONS

Interpreting findings and their impact on software quality

1 • What did fuzzing reveal that normal examples missed?

The starter assumed that the PAN, amount, and slab input was always a correctly formatted string. Generated None values caused AttributeError; malformed currency text caused ValueError; missing slab separators caused unpacking errors. Additional deterministic examples exposed invalid comma grouping, reversed slab bounds, and tax rates beyond 100%.

2 • Why does property-based fuzzing complement unit tests?

Traditional examples check a few anticipated values. Hypothesis automatically varies text, whitespace, Unicode, empty strings, and null inputs and minimizes failures to small reproducible counterexamples. We combined no-crash properties with precise valid-input round trips, so the tests measure both safety and correct functionality.

3 • What changes made the program robust, and what remains important?

Early type checks prevent attribute and split errors; anchored regular expressions reject malformed formats; bounds checks reject unreasonable values; and None distinguishes invalid amounts/slabs from valid zero. A normalizing PAN helper returns an empty string for invalid types. For a production tax service, formal PAN syntax, statutory tax rules and domain-specific error objects would need additional validation.

Submitted artifacts

income_tax_parser.py • test_income_tax_parser.py • PES1UG24AM920_ARJUN_SRIKANTH_LAB7.pdf • raw logs and screenshot images in evidence/

Run: python -m pip install -r requirements.txt; python -m pytest -q test_income_tax_parser.py